

DOFBOT × Xinu メッシュ 6 台

システム全体設計 (第 2 版)

Capability 推論による役割分担・冗長化・アクター移動

2026 年 7 月 21 日

Abstract

第 1 版は 12 台を静的な ServoMotor/ServoSensor 対として扱い、当初構想の中核である Capability 推論による役割分担・冗長性・アクターの動的移動を設計に反映していなかった。本版はそれを中心に据えて構成し直す。

中心となる主張は 1 つ。アクターの効果集合から、配置・複製可否・フェイルオーバーの手順が機械的に決まる。純粋なアクターは自由に複製でき、外部作用を持つアクターは同時に 1 つしか存在してはならない — つまりフェイルオーバーとは Capability の移譲そのものである。

ただし現状 2 つの障害がある。効果格子が「自己状態の変更」と「外部への作用」を同一視していること、そして実測レイテンシの裾が秒単位であること。前者は型システムの小さな拡張で解ける。後者は解けないので、フェイルオーバーは秒オーダーであるという前提で安全性を設計する。

1 当初構想と第 1 版のずれ

当初の構想は次のとおりであった。

AIPL のアクターを生成し、Capability 推論で役割分担させ、6 軸に対して 2 台ずつ Xinu を用意して冗長性を持たせ、アクターの移動を動的に行い、全体として効率の良いリアルタイム OS とする。

第 1 版はこのうち「6 軸 × 2 台」だけを写し取り、残りを落としていた。Capability は役割の説明に使われるだけで設計判断に効いておらず、冗長性は台数の話に留まり、アクター移動に至っては触れていない。本版で立て直す。

2 現物 — CG シミュレータ

図 1 が本設計の出発点である。重要なのは右下のパネルで、効果はソースに書かれたものではなく推論結果であり、実処理系の --type-check が申告と実体の一致を検査する。本設計はこのパネルが示す情報だけから配置・複製・フェイルオーバーを導く。

シミュレータは 6 軸それぞれに soft-Xinu を 2 台、計 12 個のアクターを走らせている。実機 6 台は、このうち 6 個を置き換えるという位置づけになる。

3 6 台をどう割り当てるか

6 台の割当には 2 通りある。

案	構成	評価
A	6 関節 × 1 台	アーム全軸が実機 Xinu になる。ただし冗長性がゼロで、当初構想の中核であるフェイルオーバーとアクター移動を実証できない
B	3 関節 × 2 台	3 つの独立したリソースで冗長化が成立することを示せる。残る 3 関節は CG 側の soft-Xinu が担う



Figure 1: DOFBOT ライン作業シミュレータ (:8022)。2026-07-21 実行、node test/verify.mjs で全項目合格した状態。左は 3D の作業セル (銀色ホーンの丸数字が実機サーボ ID 1-6 に対応)、右上は手首カメラの映像と TinyML の分類結果 (緑・六角柱 100%)、中央右は実行中の AIPL ソースで、現在実行している行が強調表示される (PlannerActor 268 行目)、右下が Actor / Capability パネルで、各アクターの効果推論結果 `!{mut} !{net}` がチップとして表示される。

案 B を採る。 理由は、実証したい主張が「アームが動くこと」ではなく「**Capability に基づく役割分担・冗長化・動的移動が成立すること**」だからである。案 A では冗長性が 1 つも無く、リース移譲も昇格も試験できない。一方、案 B で足りない 3 関節は CG が肩代わりするので、アームとしては 6 軸すべてが動く。ここがシミュレータを持っていることの利点である。

実機に載せる 3 関節は ID1 (ベース旋回)・ID2 (肩)・ID3 (肘) とする。この 3 つが搬送時の関節変位の大半を占め、サイクル時間を支配するため、実時間性の評価対象として意味がある。ID4-ID6 は CG 側に残す。

ノード	担当	役割	保持する効果
1	ID1	ServoMotor (主)	!{act, mut, net}
2	ID1	ServoSensor (副)	!{net}
3	ID2	ServoMotor (主)	!{act, mut, net}
4	ID2	ServoSensor (副)	!{net}
5	ID3	ServoMotor (主)	!{act, mut, net}
6	ID3	ServoSensor (副)	!{net}

台数効果の測定点は 2 / 4 / 6 の 3 点になる。1 関節対・2 関節対・3 関節対と段階的に増やせば、同じ作業に対するサイクル時間の傾向が取れる。1 点では傾向を主張できないので、6 台という数は「3 点が取れる最小」でもある。

純粹アクター (Kinematics) は 6 台すべてに複製する。効果を持たないので二重実行が無害であり、冗長化の費用がゼロだからである (第 4.3 節)。

4 Capability から配置と冗長化を導く

4.1 効果は宣言ではなく推論される

AIPL の Capability はソースに書くものではない。各メソッド本体の一次作用を効果推論が走査して !{mut, net, ai, fs} を導出し、--type-check が申告と実体の食い違いを検出する。現在の割当は次のとおり。

一次作用	効果	意味
Arm_serial_servo_write	mut	実機バスサーボの駆動
Arm_serial_servo_read	なし	計測のみ。何も動かさない
tinymml_infer	ai	ローカル推論。net を伴わない = 機外へ出ない
tinymml_load / write_file	fs	記憶装置
send / now / reply	net	メッセージ

これは強力である。ServoSensor は Arm_serial_servo_write を呼ぶ場所がソース中のどこにも無いため、**型によって物理的に駆動できないことが保証される**。最小権限が設計意図ではなく検査可能な性質になっている。

4.2 発見: 効果格子が作用と状態変更を同一視している

しかし配置の自動導出には足りない。現在のクラスの効果は次のようになる。

クラス	効果	外部への作用
ServoMotor	!{mut, net}	あり (サーボを回す)
ShelfActor	!{mut, net}	なし (自分の在庫表を書くだけ)
ConveyorActor	!{mut, net}	あり (ストッパを開閉)
ArmMover	!{net}	なし
Kinematics	!{net}	なし (純計算 + reply)
RecognitionActor	!{ai, net}	なし
SafetyActor	!{fs, mut, net}	なし

ServoMotor と ShelfActor は効果集合が同一である。 一方は実機を動かし、他方は自分のフィールドを書くだけなのに区別が付かない。ソース中のコメント自身が「bind() の mut は自分のフィールド id への代入であって、サーボ駆動ではない」と断っており、この曖昧さは認識されている。

配置と複製の規則を効果から導くには、この 2 つを分ける必要がある。

提案する効果	意味	導かれる規則
mut act	自分の状態を書く 外部世界へ作用する	複製可（各複製が自分の状態を持つ） 同時に 1 つだけ 。配置はその機器に繋がったノードに限定

Arm_serial_servo_write と conveyor_stopper を act に移すだけで、ServoMotor/ConveyorActor と ShelfActor が型で分かれる。これは小さな変更だが、以降の設計全体がこの区別の上に載る。

4.3 効果集合から決まる規則

効果	配置	複製	根拠
なし（純粹）	どこでも	自由	副作用が無いので二重実行が無害。冗長化の費用がゼロ
!{net}	どこでも	可（冪等なら）	重複メッセージを受け手が吸収できる必要
!{mut}	どこでも	可	各複製が独立した状態を持つ
!{ai}	重みを持つノード	可	推論は決定的。機外へ出ない
!{fs}	記憶装置を持つノード	不可	書き込みが競合する
!{act}	その機器に繋がったノード	不可（厳密に 1 つ）	二重駆動は物理的に危険

Kinematics は効果を持たないので全ノードに複製してよい。逆運動学を 6 台のどこで解いても同じ答えが出るので、冗長化は無料である。一方 ServoMotor は act を持つので**厳密に 1 つ**でなければならない。

5 冗長化 — 3 関節 × 2 台

5.1 対の構造

実機を載せる 3 関節それぞれに 2 台を割り当てる。当初構想どおりだが、役割を「モータ役とセンサ役」ではなく「**act 保持者と待機者**」として定義する。

ノード	保持	役割
Xinu-A（主）	ServoMotor !{act,mut,net}	目標角を受けてサーボを駆動
Xinu-B（副）	ServoSensor !{net}	同じ関節を計測し、主の状態を鏡像として保持

副が主の状態を常に持っている点が要である。昇格時にネットワーク越しの状態転送が要らない。今日の実測ではメッシュの裾が秒単位あるため、「故障を検知してから状態を取りに行く」設計は成立しない。

5.2 昇格 = Capability の移譲

act は厳密に 1 つという規則があるので、昇格は**排他的な移譲**でなければならない。分断時に主副が同時に駆動する事態は物理的に危険である。そこで act を**期限付きリース**として扱う。

1. 主は周期 T_r でリースを更新する。
2. 主が T_e 以内に更新できなければ、主は自力で駆動を停止する（ネットワーク不要。ローカルの時計だけで判定できる）。
3. 副は T_e 経過を観測してから act を取得し、昇格する。

手順 2 が安全性の要である。 分断された主が「自分はまだ主だ」と思い込んで駆動し続けることを、ネットワークに頼らず防ぐ。これは分散ロックのリースと同じ論法で、**Capability** がリソースではなく期限付き権限であることを利用している。

5.3 タイミング — 秒オーダーであることを認める

T_e の下限は実測で決まる。

量	値	根拠
HELLO 周期	2s	device/wifi/wifi.c (現行実装)
往復レイテンシ p99	84-3009ms	本日実測 (Pi 5 / Pi 3)
リース更新 T_r	≥ 1 s	p99 の裾を吸収する
リース期限 T_e	≥ 3 s	T_r の 3 倍。誤検知を避ける
フェイルオーバー所要	3-5 s	上記の合計

したがってこの冗長化は「可用性」のためであって「無停止」のためではない。 3-5 秒のあいだ、その関節には駆動権限を持つ者がいない。その間アームが安全であることは、メッシュではなく L0 (アーム内蔵 Pi) が保証しなければならない — 最後の目標角を保持するか、停止する。これは選択肢ではなく要件である。

6 アクターの動的移動

6.1 既にあるもの・無いもの

要素	状態	備考
コード移動	実装済	/actor/load (AIPL $\rightarrow$ C JIT) が Pi 4 / Pi 5 にある
効果推論	実装済	--type-check で申告と実体を照合
act の分離	未	第 2.2 節の提案
状態の直列化	未	移動にはアクターのフィールドを運ぶ必要がある
リース機構	未	act の排他移譲
配置決定器	未	効果 + 容量から配置を解く

コード移動が既にあることは大きい。移動の難所は通常コード配布であり、そこは AIPL $\rightarrow$ C JIT と /actor/load で解決済みである。

6.2 移動の契機と手順

契機	手順
ノード故障	リース期限切れ $\rightarrow$ 副が昇格 (第 3.2 節)
負荷偏り	純粋アクターを空きノードへ複製し、要求を分散。状態が無いので停止不要
RT 締切超過	期限に間に合っていないアクターを、余裕のあるノードへ移す
熱・電源	同上

効果集合が移動の難易度をそのまま決める。純粋アクターは複製するだけでよく、停止も同期も要らない。mut を持つものは状態を運ぶ必要がある。act を持つものはリース移譲を伴い、最も高くつく。したがって移動の計画は、効果の軽いものから順に動かすのが合理的である。

7 リアルタイム性

「全体として効率の良いリアルタイム OS」にするために、どのアクターが実時間制約を持つかを明確にする。

アクター	周期	締切	置き場所
ServoMotor	10 Hz	硬	RT スケジューラを持つノード
ServoSensor	10 Hz	軟	同上（対の相方）
ArmMover	10 Hz	軟	任意
Kinematics	要求時	軟	任意（複製可）
RecognitionActor	撮像時	軟	重みを持つノード
SafetyActor	事象時	硬	記憶装置を持つノード

硬い締切を持つのは ServoMotor と SafetyActor だけである。Pi 4 のプリエンプティブ優先度スケジューラ（実測 max jitter 10 μ s、100 回中ミス 0）はこの用途に十分な性能を持つ。残りは軟らかいので配置の自由度が高く、移動の対象にしやすい。

ただし現状 RT スケジューラは Pi 4 のみで実用段階にある。 Pi 5 は移植の途中（Stage 1-2）、Pi 3 は upstream のスケジューラである。6 台構成ではどのノードが硬い締切を引き受けられるかが不均一であり、配置決定器はこれを制約として扱う必要がある。

8 学習の位置づけ

学習も「効果を持つアクターの 1 つ」として同じ枠組みで扱う。Learner は !{ai, net}（重みを読むなら fs も）であり、配置規則は第 2.3 節の表がそのまま適用される — 重みを持つノードならどこでもよく、複製も可能である。

現時点の推奨は Mac に置くことだが、これは設計ではなく性能の問題である。Xinu は 3 ボードとも **D-cache が OFF**（アセスメント項目 10、最大の性能レバーが未使用）で、ベクトル化も数値ライブラリも無い。6 台 24 コアを足しても素の Mac に勝てる見込みが薄い。**D-cache** を有効化した時点で再評価すべきであり、そのときは配置決定器に **Learner** を渡すだけで済む — 枠組みを変える必要は無い。ここが本設計の利点である。

推論（RecognitionActor、TinyML 192→12→3 = 約 2.4k 積和）は最初からメッシュ側に置く。ロボットの制御経路上にあり、計算量も小さい。

9 実装の順序

依存関係の順に並べる。各段は前段が無いと意味を持たない。

段	内容	検証
1	act 効果の分離 (Arm_serial_servo_write を mut から移す)	--type-check で ServoMotor と ShelfActor が分かれる
2	配置規則の実装 (効果集合 → 配置可能ノード集合)	単体テスト
3	状態の直列化 (アクターのフィールドを運ぶ)	移動前後で状態が一致
4	リース機構 (act の排他移譲)	分断注入で二重駆動が起きないこと
5	CG を実機メッシュに接続 (現在の 3 台で可能)	実機 Xinu が CG の関節を動かす
6	配置決定器 (効果 + 容量 + RT 能力)	故障注入で自動再配置

第 1 段は今日から着手でき、費用がほぼゼロで効果が大きい。型検査だけで検証できる（実機不要）。第 5 段は現在の 3 台で実施でき、そこで取れるサイクル時間とジッタが、さらに台数を増やす価値を判断する実数になる。

10 着手前に潰すべき前提

1. **Pi 3 の 14% 失敗。** 原因特定済（単一 accept の HTTP サーバ）、修正はコミット済み。デプロイして効果を実測する。

2. **Pi 4 の 130 ms**。有線で Pi 5 の 3.6 倍。原因未究明。
3. **時刻同期が無い**。リースの期限判定はローカル時計で足りるが、経験 (s, a, r) をノード横断で突き合わせるには共通時刻が要る。
4. **D-cache が 3 ボードとも OFF**。学習の配置判断はこれを直してから。
5. **RT スケジューラが Pi 4 のみ**。6 台の能力が不均一。ただし異種混成は「能力の異なるノード群への配置問題」として研究上の資産でもある。

11 本書の限界

- レイテンシは 200 サンプル $\times$ 3 台、1 回の測定である。
- リースの T_r / T_e は実測レイテンシからの見積もりであり、分断注入試験で検証していない。
- act 分離の提案は未実装。型検査を通していない。
- 台数効果は外挿であり、実測は 3 台 1.87 倍（効率 62%）のみ。
- アーム内蔵 Pi 5 に ROS が載っているという前提は未確認。

実測は *Raspberry Pi 3 B+ / 4 / 5* 上の *Xinu*、*MANET* は *WiFi ad-hoc (IBSS)*、*SSID MANET*、*ch 6*、*10.0.0.2/3/5*。アクター定義は `~/aipl_line_simulator/aipl/dofbot_xinu.abcl`。